

1 Overview

This lab implements a fixed-point hardware approximation of $\exp(x)$ using a 5th-order Taylor series. We compare three architectures: a baseline fully-parallel datapath, a pipelined version that inserts registers for higher frequency, and a shared-hardware FSM that reuses a single multiplier/adder to reduce DSP usage. Inputs are in Q2.14 format and outputs are in Q7.25 format.

2 Block Diagrams and Design Description

The baseline design evaluates the polynomial in one cycle with a fully-parallel multiply-add tree; control is simple valid/ready with no internal state and a single-cycle latency. The pipelined design partitions the chain into multiple stages with registers that carry partial sums and the input x , enabling one result per cycle after filling the pipeline at the cost of higher latency and more registers. The shared-hardware design uses an FSM to time-multiplex one multiplier and one adder across five Horner steps, trading throughput for much lower DSP usage; latency is multiple cycles per output.

2.1 Pipelined Circuit

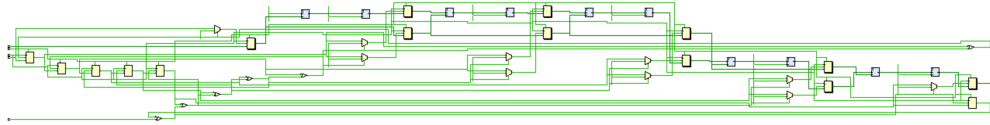


Figure 1: Block diagram of the pipelined implementation

We split the original chain by adding pipeline stages / registers between each multiplier-adder group, or to be more specific, there are four groups more of registers. For the pipeline registers, they need not only carries valid/ready logic, but also the output partial results and the input x that corresponding to this stage.

Here we briefly discuss how valid/ready propagate:

```
assign ready_5 = (~o_valid) | i_ready;
assign ready_4 = (~valid_4) | ready_5;
assign ready_3 = (~valid_3) | ready_4;
assign ready_2 = (~valid_2) | ready_3;
assign ready_1 = (~valid_1) | ready_2;
assign o_ready = (~valid_0) | ready_1;

// For simplicity, only showing one stage of valid propagate:
always @(posedge clk or posedge reset) begin
    if (reset) begin
        valid_1 <= 0;
    end else
        if (ready_1) begin
            valid_1 <= valid_0;
        end
    end
end
```

2.2 Shared-Hardware Circuit

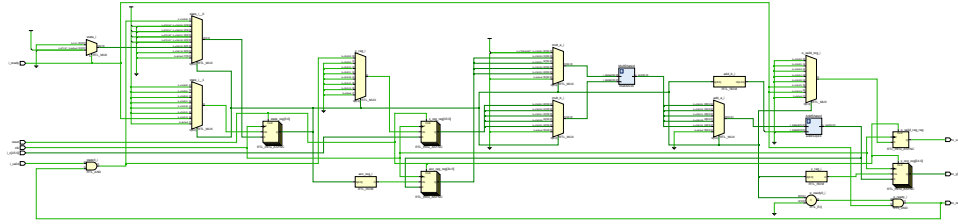


Figure 2: Block diagram of the shared-hardware implementation.

The shared-hardware circuit uses a 7-state FSM (ST_IDLE, ST_STEP0...ST_STEP4, ST_DONE) to run Horner evaluation serially. In ST_IDLE, the design accepts input when `i_valid && o_ready`, latches `i_x` into `x_reg`, then executes five compute steps, where each state reuses the same `mult32x16` and `addr32p16` pair to compute one term update ($acc = acc * x + A_k$); ST_STEP0 starts from A5 and adds A4, then ST_STEP1 to ST_STEP4 add A3, A2, A1, and A0. The partial result is stored in `acc_reg`, the final output in `y_reg`, and ST_DONE holds control until `i_ready` so the result handshake can complete. This schedule trades throughput for area by replacing multiple parallel arithmetic blocks with one shared multiplier and one shared adder.

3 Simulation and Functional Verification

3.1 Pipelined Circuit Simulation

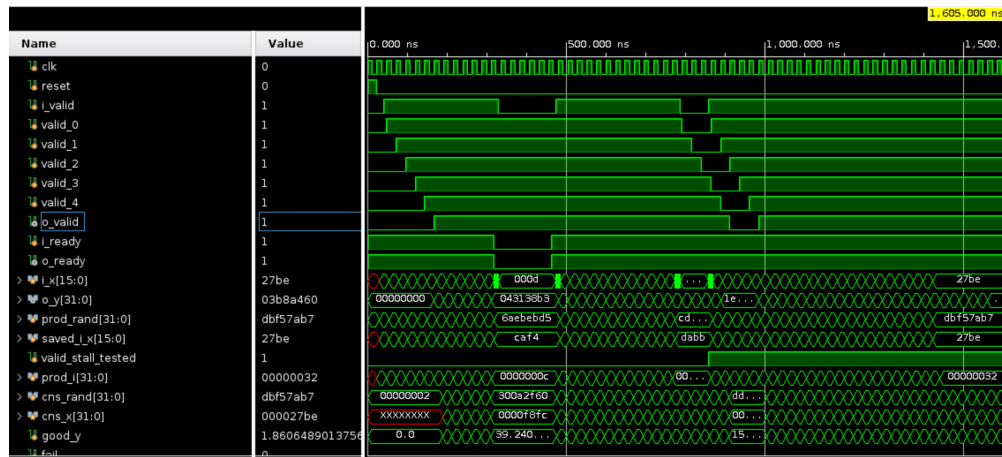


Figure 3: Pipeline Simulation Waveform

As seen in above pipeline simulation, we can clearly see the propagation of valid signal from `i_valid` to `valid_*`, all the way to `o_valid`.

Get ALL TESTS PASSED from the testbench.

3.2 Shared-Hardware Circuit Simulation

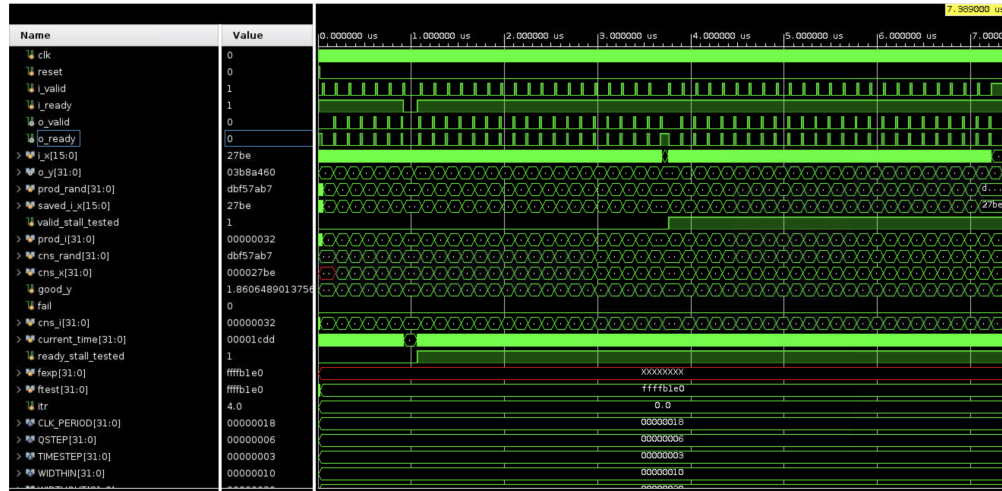


Figure 4: Waveform showing correct valid/ready behavior and outputs.

As seen in above shared-hardware simulation, the `o_valid` and `o_ready` are set active at most every 6 cycles for one cycle, which corresponds to the behavior of shared design. Get ALL TESTS PASSED from the testbench.

4 Implementation Results

Table 1 summarizes timing, utilization, and power from Vivado post-implementation reports. Operating frequency is taken from the timing summary (WNS/achievable Fmax), resource counts from utilization reports, and dynamic power from the power report for a single instance.

Table 1: Implementation results for the three circuit implementations.

	Baseline Circuit	Pipelined Circuit	Shared HW Circuit
Resources for one circuit	10 LUTs + 50 FFs + 7 DSPs	24 LUTs + 236 FFs + 9 DSPs	56 LUTs + 84 FFs + 2 DSPs
Operating frequency	81 MHz	250 MHz	250 MHz
Critical path	2x 1DSP mult + LUT-based adder + DSP mult + 2x 2DSP mult + LUT-based adder	2DSP mult + LUT-based Adder	1x 2DSP mult + LUT-based adder
Cycles per valid output	1	1	6
Max. # of copies/device	260 (DSP constraints)	202 (DSP constraints)	912 (DSP constraints)
Max. throughput for a full device (computations/s)	$260 \times 81 \text{ M} = 21.06 \text{ G}$	$202 \times 250 \text{ M} = 41.41 \text{ G}$	$912 \times 250 \text{ M} / 6 = 38 \text{ G}$
Dynamic power of one circuit	5 mW	11 mW	7 mW
Max. throughput/Watt for full device	12.02 G/W	15.49 G/W	5.56 G/W

5 Error Analysis and Discussion

5.1 Approximation Error

Figure 5 shows the error between the ideal $\exp(x)$ and the fixed-point hardware output over the tested input range. The dominant error sources are: (1) truncation of the Taylor series to 5th order, which grows for larger $|x|$; (2) fixed-point quantization of intermediate products and coefficients; and (3) rounding/truncation at the output formatting stage. To reduce the error, the most direct option is increasing the Taylor order (more terms) or using a piecewise approximation (range reduction followed by a low-order polynomial). Accuracy can also improve by widening intermediate bit-widths and using rounding instead of truncation, at the cost of extra DSPs and registers. State the observed max error and where it occurs in your plot.

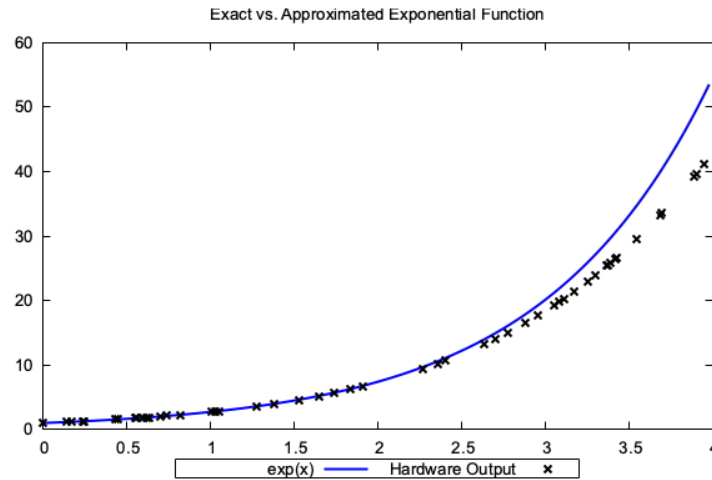


Figure 5: Error between $\exp(x)$ and hardware output across the input range.

5.2 Throughput and Efficiency Comparison

The pipelined design achieves the highest throughput per device because it sustains one output per cycle at 250 MHz while still allowing over 200 copies before hitting DSP limits. The baseline runs at a lower frequency (81 MHz) but also produces one result per cycle, so its total throughput is smaller despite a similar DSP bound. The shared-hardware design reaches 250 MHz but requires 6 cycles per output, so total throughput is lower even though it allows many more copies due to low DSP usage.

6 Conclusion

The baseline design is the simplest and lowest-latency but limited by frequency. Pipelining improves Fmax and overall throughput at the cost of additional registers and DSPs. The shared-hardware FSM minimizes DSP usage and scales to many copies but sacrifices per-instance throughput due to multi-cycle evaluation. The choice depends on whether peak throughput or area efficiency is the primary constraint.